IRO – Issues Reporting Outlet: Object Listing

CSC426 Weekly Dev – Weekend 3 · Joshua Omolafe · Matric No. 220404244

IRO is written in plain JavaScript as a small set of singleton objects. Each page has one controller object, and all of them share the configuration object `IRO` and the data-layer object `IssueStore`. The tables below list every object with its properties and methods.

1. Object summary

Object	File	Kind	Used by	Purpose
`IRO`	js/store.js	Configuration	All pages	Constant lists and settings.
`IssueStore`	js/store.js	Data layer	All controllers	Create, read, update and delete issues; statistics.
Helper functions	js/store.js	Utilities	All controllers	Escaping, date formatting, badges, filling selects.
`ReportForm`	js/report.js	Controller	report.html	Report submission.
`Tracker`	js/track.js	Controller	track.html	Status tracking by ticket ID.
`AdminDesk`	js/admin.js	Controller	admin.html	Desk officer dashboard.
Issue	(data record)	Entity	IssueStore	One reported issue stored in localStorage.

2. IRO (configuration object)

Property	Type	Value / description
`STORAGE_KEY`	String	"iro_issues" – localStorage key holding all issues.
`ADMIN_PASSCODE`	String	"iro2026" – demo passcode for the admin desk.
`CATEGORIES`	Array<String>	10 issue categories (Security / Safety … Other).
`COMMUNITIES`	Array<String>	Akungba, Iwaro, Etioro, Ayegunle, Oke-Oka, Other.
`STATUSES`	Array<String>	Submitted, Under Review, Escalated, Resolved, Closed.
`SEVERITIES`	Array<String>	Low, Medium, High, Emergency.

3. IssueStore (data-layer object)

Method	Returns	Description
`getAll()`	Array<Issue>	Reads all issues from localStorage, newest first; returns [] if data is missing or corrupt.
`saveAll(list)`	void	Writes the whole list back to localStorage as JSON.
`generateId()`	String	Builds a unique ticket ID IRO-YY-XXXXX from an unambiguous alphabet.
`create(data)`	Issue	Builds an Issue from form data (drops name/contact if anonymous), sets status Submitted with first history entry, saves it.
`findById(id)`	Issue | null	Case-insensitive lookup by ticket ID.
`updateStatus(id, status, note)`	Issue | null	Sets the new status and appends {status, note, at} to history.
`remove(id)`	void	Deletes an issue.

`stats()`	Object	{ total, open, emergency, byStatus } for the dashboard.

4. Helper functions (global)

Function	Returns	Description
`escapeHtml(str)`	String	Replaces & < > " ' with HTML entities to prevent XSS.
`formatDate(iso)`	String	Formats an ISO date, e.g. "7 Jul 2026, 14:05".
`statusBadge(status)`	String (HTML)	Coloured badge markup for a status.
`fillSelect(select, items, placeholder)`	void	Fills a <select> with option elements.

5. ReportForm (controller for report.html)

Member	Type	Description
`form`	HTMLFormElement	The #reportForm element.
`el(id)`	Function	Shortcut for document.getElementById.
`init()`	Method	Fills drop-downs, sets the max date, attaches event listeners.
`toggleReporterFields()`	Method	Shows matric field for students, community list for indigenes.
`toggleIdentity()`	Method	Hides name/contact when Anonymous is ticked.
`collect()`	Method → Object	Reads every form field into a plain object.
`validate(d)`	Method → String	Returns an error message, or "" if valid.
`submit(e)`	Method	Validates, calls IssueStore.create(), shows the ticket ID.

6. Tracker (controller for track.html)

Member	Type	Description
`el(id)`	Function	Shortcut for document.getElementById.
`init()`	Method	Attaches the form handler; auto-tracks an `?id=` in the URL.
`show(id)`	Method	Finds the issue and renders title, meta, status and history (no personal data).

7. AdminDesk (controller for admin.html)

Member	Type	Description
`SESSION_KEY`	String	"iro_admin" – sessionStorage flag for a logged-in officer.
`selectedId`	String | null	Ticket ID of the report open in the detail panel.
`el(id)`	Function	Shortcut for document.getElementById.
`init()`	Method	Fills filters, attaches listeners, restores an existing session.
`login(e)` / `logout()`	Method	Checks passcode and opens the dashboard / ends the session.
`showDashboard()`	Method	Hides login card, shows dashboard, renders data.

`filtered()`	Method → Array	Issues matching search text and status/category/reporter filters.
`render()`	Method	Updates statistics cards and the report table.
`openDetail(id)`	Method	Shows full report details, reporter identity and history.
`saveUpdate(e)`	Method	Calls IssueStore.updateStatus() with the chosen status and note.
`deleteSelected()`	Method	Confirms, then deletes the open report.

8. Issue (data record)

Property	Type	Description
`id`	String	Ticket ID.
`reporterType`	String	Student | Indigene | Other.
`anonymous`	Boolean	Reporter chose anonymity.
`name`, `contact`	String	Empty when anonymous.
`matricNo`, `community`	String	Student matric number / indigene community.
`category`, `severity`	String	From IRO.CATEGORIES / IRO.SEVERITIES.
`location`, `incidentDate`	String	Where and when (YYYY-MM-DD).
`title`, `description`	String	Summary and full account.
`status`	String	Current status.
`createdAt`	String (ISO)	Submission time.
`history`	Array<{status, note, at}>	Append-only status log.

9. HTML page objects (main DOM elements)

Page	Key element IDs
index.html	#categoryList
report.html	#reportForm, #reporterType, #matricGroup, #matricNo, #communityGroup, #community, #anonymous, #identityGroup, #name, #contact, #category, #severity, #location, #incidentDate, #title, #description, #formError, #formCard, #successCard, #ticketId, #trackLink
track.html	#trackForm, #ticketInput, #trackError, #resultCard, #resTitle, #resMeta, #resStatus, #resHistory
admin.html	#loginCard, #loginForm, #passcode, #loginError, #dashboard, #logoutBtn, #statTotal, #statOpen, #statEmergency, #statResolved, #search, #filterStatus, #filterCategory, #filterReporter, #issueRows, #emptyMsg, #detailCard, #dTitle, #dMeta, #dReporter, #dDesc, #dHistory, #updateForm, #newStatus, #note, #deleteBtn